

6. Linear Algebra

6.1. Introduction and Nomenclature

Linear algebra is one of the cornerstones of theoretical and numerical physics. Whether in quantum mechanics, classical mechanics, electrodynamics, or statistical physics, we frequently work with systems of equations, operators, and transformations — all of which are naturally described using vectors and matrices.

6.1.1. Vectors

A **vector** is an element of a vector space. In physics, vectors often represent quantities with magnitude and direction, such as displacement, velocity, and force.

In linear algebra, a vector in $\mathbb{R}^n$ is an ordered n -tuple of real numbers:

$$\vec{v} = \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{pmatrix} \in \mathbb{R}^n$$

Key operations we typically perform with vectors are

- **Vector addition:** $\vec{u} + \vec{v}$,
- **Scalar multiplication:** $\alpha \vec{v}$ for $\alpha \in \mathbb{R}$ and
- **Dot product:** $\vec{u} \cdot \vec{v} = \sum_{i=1}^n u_i v_i$.

6.1.2. Matrices

A **matrix** is a rectangular array of numbers:

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix} \in \mathbb{R}^{m \times n}.$$

Matrices can represent linear transformations, systems of equations, and operators. In physics, matrices are crucial in:

- Quantum mechanics (e.g., Hamiltonians)

6. Linear Algebra

- Rotations and Lorentz transformations

Matrix operations include

- **Matrix multiplication:** AB

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}, \quad B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}$$

$$A \cdot B = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{bmatrix}$$

- **Transpose:** A^T

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix} \Rightarrow A^T = \begin{bmatrix} a_{11} & a_{21} \\ a_{12} & a_{22} \\ a_{13} & a_{23} \end{bmatrix}$$

- **Inverse:** A^{-1} (if it exists, see later)

6.1.3. Matrix Equations

A typical linear system of equations can be written as:

$$A\vec{x} = \vec{b}$$

where A is an $m \times n$ matrix, $\vec{x} \in \mathbb{R}^n$ is the vector of unknowns, and $\vec{b} \in \mathbb{R}^m$ is a known vector.

Solving this system involves:

- Inverting A (if A is square and invertible): $\vec{x} = A^{-1}\vec{b}$
- Using Gaussian elimination
- Applying numerical methods for large or non-square systems

Note: If $m \neq n$, the system is either overdetermined ($m > n$) or underdetermined ($m < n$), and solutions may not exist or may not be unique. In such cases, least squares methods or generalized inverses are used.

Dimensionality and the Solution Space

The dimensions of a matrix have deep implications for the nature of the linear system

$$A\vec{x} = \vec{b},$$

where $A \in \mathbb{R}^{n \times m}$, $\vec{x} \in \mathbb{R}^m$, and $\vec{b} \in \mathbb{R}^n$.

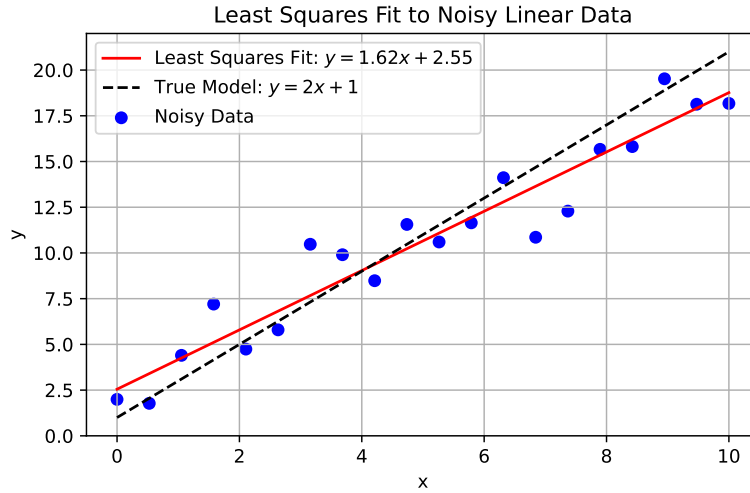


Figure 6.1.: Example of an over-determined system: Least square fitting.

Case 1: $n = m$ (Square matrix)

This is the most familiar case. The matrix has the same number of equations and unknowns.

- If $\det(A) \neq 0$, then the system has a **unique solution**.
- If $\det(A) = 0$, then the matrix is **singular**, and the system may have either **no solution** or **infinitely many solutions**.

Case 2: $m > n$ (More equations than unknowns)

This is an **overdetermined system** — there are more constraints than degrees of freedom.

- In general, such a system has **no exact solution**.
- Often, a **least-squares solution** is sought that minimizes the residual $\|A\vec{x} - \vec{b}\|$.
- Physically, this appears in data fitting and statistical models with noisy data.

Connection to Least Squares Fitting. In experimental physics and data analysis, overdetermined systems naturally arise when fitting a model to measured data, see Fig. 6.1. Suppose we want to fit a straight line $y = sx + c$ to a set of n data points (x_i, y_i) with $m > 2$. The fitting conditions can be written as a linear system:

$$A\vec{x} = \begin{pmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_m & 1 \end{pmatrix} \begin{pmatrix} s \\ c \end{pmatrix}, \quad \vec{b} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{pmatrix} \Rightarrow A\vec{x} \neq \vec{y}$$

6. Linear Algebra

Here, $A \in \mathbb{R}^{m \times 2}$ is the design matrix, and $\vec{x} = (s, c)^\top$ contains the parameters. Since $m > 2$, this system is overdetermined. An exact solution is unlikely to exist due to measurement noise. Instead, the best-fit parameters minimize the squared error $\|A\vec{x} - \vec{y}\|^2$, which leads to the **normal equations**:

$$A^\top A \vec{x} = A^\top \vec{y}$$

Solving these gives the least-squares estimate for $\vec{x}$. This method underpins linear regression and many fitting procedures in physics and engineering.

Case 3: $n > m$ (More unknowns than equations)

This is an **underdetermined system** — there are fewer constraints than variables.

- In general, there are **infinitely many solutions**.
- Additional criteria (such as minimal norm) may be imposed to find a unique solution.

Physical Interpretation

- In mechanics, an overdetermined system may imply incompatible constraints.
- In quantum mechanics, underdetermined systems relate to degeneracies or symmetries.

6.1.4. Determinant

The **determinant** is a scalar value that can be computed from a square matrix and provides important information about the matrix. For a matrix $A \in \mathbb{R}^{n \times n}$, the determinant is written as $\det(A)$ or $|A|$.

Definition for 2×2 and extension to $n \times n$

2x2 Matrix

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \Rightarrow \det(A) = ad - bc$$

3x3 Matrix (Cofactor Expansion)

$$\det \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = a_{11} \begin{vmatrix} a_{22} & a_{23} \\ a_{32} & a_{33} \end{vmatrix} - a_{12} \begin{vmatrix} a_{21} & a_{23} \\ a_{31} & a_{33} \end{vmatrix} + a_{13} \begin{vmatrix} a_{21} & a_{22} \\ a_{31} & a_{32} \end{vmatrix}$$

General $n \times n$ Matrix For $A = [a_{ij}] \in \mathbb{R}^{n \times n}$,

$$\det(A) = \sum_{j=1}^n (-1)^{i+j} a_{ij} \cdot \det(A_{ij}),$$

where A_{ij} is the minor matrix formed by removing row i and column j .

Properties

- $\det(AB) = \det(A) \det(B)$
- $\det(A^\top) = \det(A)$
- $\det(I) = 1$
- If $\det(A) = 0$, then A is not invertible.

Geometric Interpretation

The determinant gives the scaling factor by which the linear transformation described by A changes areas (in 2D) or volumes (in 3D). A zero determinant means the transformation flattens the space into a lower dimension (i.e., the matrix is **singular** and not invertible). If, for instance, $\det(A) = 2$, this means that the transformation doubles the volume/area/hypervolume.

Functional Determinant (Jacobian)

When changing variables in nonlinear coordinate systems — such as switching from Cartesian to polar or spherical coordinates — we use the *Jacobian determinant* to account for how the volume element transforms.

Example 1: Polar Coordinates in 2D Let

$$x = r \cos \theta, \quad y = r \sin \theta.$$

Then the Jacobian matrix is:

$$J = \begin{bmatrix} \frac{\partial x}{\partial r} & \frac{\partial x}{\partial \theta} \\ \frac{\partial y}{\partial r} & \frac{\partial y}{\partial \theta} \end{bmatrix} = \begin{bmatrix} \cos \theta & -r \sin \theta \\ \sin \theta & r \cos \theta \end{bmatrix}$$

and the determinant is

$$\det(J) = r.$$

Hence, the area element transforms as:

$$dx dy = \det(J) dr d\theta = r dr d\theta.$$

6. Linear Algebra

Example 2: Spherical Coordinates in 3D The transformation to spherical coordinates is:

$$x = r \sin \theta \cos \phi,$$

$$y = r \sin \theta \sin \phi,$$

$$z = r \cos \theta.$$

The Jacobian determinant is:

$$\det(J) = r^2 \sin \theta,$$

so the volume element becomes:

$$dx dy dz = r^2 \sin \theta dr d\theta d\phi.$$

General Case In general, when transforming coordinates $\vec{x} = \vec{x}(u_1, \dots, u_n)$, the integral becomes:

$$\int_{\mathbb{R}^n} f(\vec{x}) d\vec{x} = \int_{\Omega} f(\vec{x}(u_1, \dots, u_n)) \left| \det \left(\frac{\partial \vec{x}}{\partial (u_1, \dots, u_n)} \right) \right| du_1 \dots du_n.$$

Determinant of Rotation Matrices

Rotation matrices are orthogonal matrices with determinant 1, i.e. they preserve the length of the vector.

In 2D:

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \Rightarrow \det(R) = \cos^2 \theta + \sin^2 \theta = 1$$

In n D: A rotation matrix $R \in \mathbb{R}^{n \times n}$ satisfies:

$$R^T R = I \quad \text{and} \quad \det(R) = 1$$

Such matrices form the *special orthogonal group* $SO(n)$.

Note: Orthogonal matrices with $\det = -1$ represent reflections or improper rotations and are not considered pure rotations.

6.1.5. Orthogonal matrices and orthonormal vectors

Orthogonality plays a central role in numerical linear algebra because orthogonal transformations preserve lengths, angles, and numerical stability.

Orthogonal vectors

Two vectors $u, v \in \mathbb{R}^n$ are called orthogonal if their scalar product vanishes

$$u^T v = 0. \quad (6.1)$$

A set of vectors $\{q_1, \dots, q_n\}$ is called orthonormal if

$$q_i^T q_j = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases} \quad (6.2)$$

Thus, orthonormality combines two properties

- orthogonality:

$$q_i^T q_j = 0 \quad (i \neq j), \quad (6.3)$$

- normalization:

$$q_i^T q_i = 1. \quad (6.4)$$

Geometrically, orthonormal vectors form perpendicular unit vectors and therefore define a convenient coordinate basis.

Orthogonal matrices

A square matrix

$$Q \in \mathbb{R}^{n \times n} \quad (6.5)$$

is called orthogonal if

$$Q^T Q = Q Q^T = I. \quad (6.6)$$

This immediately implies

$$Q^{-1} = Q^T. \quad (6.7)$$

An orthogonal matrix can be written in terms of its column vectors,

$$Q = \begin{pmatrix} | & | & \cdots & | \\ q_1 & q_2 & \cdots & q_n \\ | & | & \cdots & | \end{pmatrix}. \quad (6.8)$$

Using

$$Q^T Q = \begin{pmatrix} q_1^T q_1 & q_1^T q_2 & \cdots \\ q_2^T q_1 & q_2^T q_2 & \cdots \\ \vdots & \vdots & \ddots \end{pmatrix} = I, \quad (6.9)$$

one sees that the columns of an orthogonal matrix form an orthonormal basis.

Length preservation

One of the most important properties of orthogonal matrices is that they preserve vector lengths

$$\|Qx\|_2 = \|x\|_2. \quad (6.10)$$

This follows directly:

$$\|Qx\|_2^2 = (Qx)^T(Qx) = x^T Q^T Qx = x^T x = \|x\|_2^2. \quad (6.11)$$

Therefore orthogonal transformations correspond geometrically to pure rotations and reflections without stretching. This property is extremely important in numerical computations because orthogonal matrices do not amplify numerical errors.

6.1.6. Eigenvalues and Eigenvectors

One of the most powerful concepts in linear algebra is that of **eigenvalues** and **eigenvectors**.

Given a square matrix $A \in \mathbb{R}^{n \times n}$, a non-zero vector $\vec{v}$ is an eigenvector of A if

$$A\vec{v} = \lambda\vec{v}$$

for some scalar λ , called the **eigenvalue**.

Finding Eigenvalues

To find eigenvalues, solve the **characteristic equation**:

$$\det(A - \lambda I) = 0$$

This yields a polynomial equation of degree n in λ .

Example

Let

$$A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$$

The characteristic polynomial is:

$$\det(A - \lambda I) = \begin{vmatrix} 2 - \lambda & 1 \\ 1 & 2 - \lambda \end{vmatrix} = (2 - \lambda)^2 - 1 = \lambda^2 - 4\lambda + 3$$

The eigenvalues are $\lambda_1 = 1$, $\lambda_2 = 3$.

Eigenvalues and Eigenvectors in Physics

Eigenvalues and eigenvectors appear in many areas of physics, often revealing intrinsic properties of a system such as symmetry, stability, or energy levels.

1. Quantum Mechanics: Operators and Observables

In quantum mechanics, measurable quantities (observables) are represented by Hermitian operators. The eigenvalues of an operator correspond to the possible results of a measurement, and the eigenvectors describe the quantum states with definite values of the observable.

Example: Spin Operator $\hat{S}_z$ The spin operator for a spin- $\frac{1}{2}$ particle along the z -axis is

$$\hat{S}_z = \frac{\hbar}{2} \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}.$$

We solve the eigenvalue equation

$$\hat{S}_z \vec{v} = \lambda \vec{v}.$$

The characteristic equation is

$$\det(\hat{S}_z - \lambda I) = \left(\frac{\hbar}{2} - \lambda\right) \left(-\frac{\hbar}{2} - \lambda\right) = 0,$$

which gives the eigenvalues

$$\lambda = \pm \frac{\hbar}{2}.$$

The corresponding eigenvectors are

$$\vec{v}_+ = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \vec{v}_- = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

These represent the spin-up and spin-down states in the z -direction.

2. Classical Mechanics: Normal Modes of Oscillation

In classical mechanics, small oscillations of coupled systems can be analyzed by diagonalizing the dynamical matrix. The eigenvectors represent the *normal modes* of motion, and the eigenvalues give the *squared angular frequencies* of these modes.

Example: Two Masses and Three Springs Two masses m are connected by three springs of constant k :

$$\text{Wall} \leftrightarrow m_1 \leftrightarrow m_2 \leftrightarrow \text{Wall}$$

The equations of motion are

$$m\ddot{x}_1 = -2kx_1 + kx_2, \quad m\ddot{x}_2 = -2kx_2 + kx_1.$$

6. Linear Algebra

This gives the matrix equation:

$$m\ddot{\vec{x}} = -K\vec{x}, \quad K = k \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}.$$

To find the normal modes, solve the eigenvalue problem:

$$K\vec{v} = \lambda\vec{v}.$$

The eigenvalues are

$$\lambda_1 = k, \quad \lambda_2 = 3k \quad \Rightarrow \quad \omega_1 = \sqrt{\frac{k}{m}}, \quad \omega_2 = \sqrt{\frac{3k}{m}}.$$

The corresponding eigenvectors (normalized) are:

$$\vec{v}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \vec{v}_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

These represent:

- Mode 1: both masses oscillate **in phase** (center-of-mass motion).
- Mode 2: masses oscillate **out of phase** (breathing mode).

3. Stability Analysis: Growth or Decay of Perturbations

In many systems, particularly in fluid dynamics or cosmology, one studies the evolution of small perturbations around equilibrium. These follow a linearized equation of the form:

$$\frac{d\vec{x}}{dt} = A\vec{x}.$$

The eigenvalues of A determine the behavior:

- $\text{Re}(\lambda) < 0$: perturbation decays (stable),
- $\text{Re}(\lambda) > 0$: perturbation grows (unstable),
- $\text{Im}(\lambda) \neq 0$: oscillatory behavior.

Example: 2D Linear System Let

$$A = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix}.$$

Then

$$\det(A - \lambda I) = \begin{vmatrix} -\lambda & 1 \\ -2 & -3 - \lambda \end{vmatrix} = \lambda^2 + 3\lambda + 2 = 0.$$

The eigenvalues are

$$\lambda_1 = -1, \quad \lambda_2 = -2.$$

Since both eigenvalues have negative real parts, the system is **stable**, and all perturbations decay over time.

6.1.7. Matrix Multiplication Rules

Given two matrices $A \in \mathbb{R}^{n \times m}$ and $B \in \mathbb{R}^{m \times p}$, their product $C = AB \in \mathbb{R}^{n \times p}$ is defined by

$$C_{ij} = \sum_{k=1}^m A_{ik} B_{kj}$$

Key properties of matrix multiplication:

- **Associativity:** $A(BC) = (AB)C$
- **Distributivity:** $A(B + C) = AB + AC$
- **Compatibility:** The number of columns of A must equal the number of rows of B
- **Non-commutativity:** In general, $AB \neq BA$

These rules reflect how matrix products correspond to successive linear transformations.

6.1.8. Matrix Norms

Matrix norms allow us to measure the "size" or "magnitude" of a matrix. They are used in error analysis, convergence estimates, and stability analysis.

Vector L_p Norm: For a vector $\vec{x} = (x_1, \dots, x_n)$, the L_p norm is defined as

$$\|\vec{x}\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}, \quad 1 \leq p < \infty$$

and the L_∞ norm is

$$\|\vec{x}\|_\infty = \max_i |x_i|$$

Matrix Norms (Induced Norms): A matrix norm $\|A\|_p$ can be defined as the maximum amplification of the vector L_p norm under multiplication:

$$\|A\|_p = \max_{\vec{x} \neq 0} \frac{\|A\vec{x}\|_p}{\|\vec{x}\|_p}$$

Common cases:

- $p = 2$: the **spectral norm** (largest singular value of A)
- $p = 1$: maximum absolute column sum
- $p = \infty$: maximum absolute row sum

6.1.9. Condition Number of a Matrix

The **condition number** of a matrix $A \in \mathbb{R}^{n \times n}$ with respect to a norm $\|\cdot\|$ is defined as:

$$\kappa(A) = \|A\| \cdot \|A^{-1}\|$$

It measures how sensitive the solution $\vec{x}$ to $A\vec{x} = \vec{b}$ is to small changes in $\vec{b}$ or A .

Interpretation:

- $\kappa(A) \approx 1$: well-conditioned matrix (numerically stable)
- $\kappa(A) \gg 1$: ill-conditioned matrix (small perturbations in input can cause large errors)
- $\kappa(A) = \infty$: singular matrix (not invertible)

Example: For the L_2 norm, the condition number is the ratio of the largest to smallest singular value:

$$\kappa_2(A) = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)}$$

Application in Physics: In numerical simulations, especially involving finite difference or finite element methods, high condition numbers can lead to large round-off errors. Preconditioning techniques are often used to improve conditioning.

A large condition number means that small uncertainties in $\vec{b}$ can lead to large variations in the solution vector $\vec{x}$. To see this, we follow how an error $\delta\vec{b}$ translates into an uncertainty $\delta\vec{x}$. The equation $A\vec{x} = \vec{b}$ has the formal solution $\vec{x} = A^{-1}\vec{b}$, and so the uncertainty in $\vec{b}$ leads to

$$(\vec{x} + \delta\vec{x}) = A^{-1}(\vec{b} + \delta\vec{b}).$$

Now consider the relative error $\|\delta\vec{b}\|/\|\vec{b}\|$, and see how it translates into a relative error in the solution,

$$\begin{aligned} \frac{\|\delta\vec{x}\|}{\|\vec{x}\|} &= \frac{\|A^{-1}\delta\vec{b}\|}{\|A^{-1}\vec{b}\|} \leq \frac{\|A^{-1}\| \cdot \|\delta\vec{b}\|}{\|A^{-1}\vec{b}\|} = \frac{\|A\| \cdot \|A^{-1}\| \cdot \|\delta\vec{b}\|}{\|A\| \cdot \|A^{-1}\vec{b}\|} \leq \frac{\|A\| \cdot \|A^{-1}\| \cdot \|\delta\vec{b}\|}{\|A \cdot A^{-1}\vec{b}\|} \\ &= \kappa(A) \cdot \frac{\|\delta\vec{b}\|}{\|\vec{b}\|}. \end{aligned}$$

The amplification factor is simply the condition of the matrix A .

6.2. Matrix solutions

To solve a system, we apply elementary transformations to the system, where each transformation does not alter the solution. We can formulate three elementary transformations. We denote the rows of the system as R_i , the columns C_i .

(1) Row exchange: Changing the order of the rows, $R_i \leftrightarrow R_j$, does not influence the result. This operation corresponds to multiplying the system with the following transformation matrix:

$$T_{ij} = \begin{pmatrix} 1 & & & & & \\ & \ddots & & & & \\ & & 0 & \cdots & 1 & \\ & & \vdots & \ddots & \vdots & \\ & & 1 & \cdots & 0 & \\ & & & & & \ddots & \\ & & & & & & 1 \end{pmatrix} \begin{matrix} \\ \\ \leftarrow i \\ \\ \leftarrow j \\ \\ \end{matrix}$$

$$\begin{matrix} \uparrow & \uparrow \\ i & j \end{matrix}$$

The matrix T_{ij} has the following properties. It is the inverse of itself, $T_{ij}^{-1} = T_{ij}$, its determinant $\det(T_{ij})$ is -1 , and so it follows that

$$\det(T_{ij} \cdot A) = -\det(A).$$

(2) Row multiplication: This elementary transformation takes a row and multiplies it with a constant factor $\lambda \neq 0$. The corresponding transformation matrix is:

$$T_i(\lambda) = \begin{pmatrix} 1 & & & & & \\ & \ddots & & & & \\ & & 1 & & & \\ & & & \lambda & & \\ & & & & 1 & \\ & & & & & \ddots & \\ & & & & & & 1 \end{pmatrix} \leftarrow i$$

$$\begin{matrix} \uparrow \\ i \end{matrix}$$

This is a diagonal matrix with the inverse $T_i^{-1}(\lambda) = T_i(1/\lambda)$. Furthermore, $\det(T_i(\lambda)) = \lambda$, and therefore

$$\det(T_i(\lambda) \cdot A) = \lambda \cdot \det(A).$$

(3) Row addition: Finally, we can add multiples of another row to the row of interest, $R_i \rightarrow R_i + \lambda \cdot R_j$. The transformation matrix is:

$$T_{ij}(\lambda) = \begin{pmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & \vdots & \ddots & \\ & & \lambda & \cdots & 1 \\ & & & & \ddots & \\ & & & & & 1 \end{pmatrix} \begin{matrix} \\ \\ \leftarrow i \\ \\ \leftarrow j \\ \\ \end{matrix}$$

$$\begin{matrix} \uparrow & \uparrow \\ i & j \end{matrix}$$

These transformations are shear mappings. The matrix is triangular, meaning that all entries above the diagonal are zero. Its inverse is $T_{ij}^{-1}(\lambda) = T_{ij}(-\lambda)$, and because $\det(T_{ij}(\lambda)) = 1$ we have

$$\det(T_{ij}(\lambda) \cdot A) = \det(A).$$

Row multiplication (2) is simply a special form of row addition (3) as defined above.

(4) Column exchange: Often it is desired to also exchange columns, for example when doing pivot search. When exchanging column i and j , $C_i \leftrightarrow C_j$, then we also need to exchange the corresponding rows in $\vec{x}$ and in $\vec{b}$.

Note, row operations are realized by multiplying the transformation matrices from the left, column operations by multiplying the matrices from the right.

6.3. Gauss-Jordan elimination

Let us now solve the system $A\vec{x} = \vec{b}$ using the example

$$A = \begin{bmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{bmatrix}, \quad b = \begin{bmatrix} 8 \\ -11 \\ -3 \end{bmatrix}$$

6.3.1. Transformation to a triangular matrix

We solve the system using the *augmented matrix*, which is simply $[A|\vec{b}]$. The goal is to bring the matrix into triangular shape and then back-substitute the values for x_i . (Note: The diagonal elements a_{ii} are called *pivot* elements):

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ -3 & -1 & 2 & -11 \\ -2 & 1 & 2 & -3 \end{array} \right]$$

Step 1: Eliminate below the pivot in column 1 Use row operations:

$$R_2 \leftarrow R_2 + \frac{3}{2}R_1, \quad R_3 \leftarrow R_3 + R_1$$

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ 0 & 0.5 & 0.5 & 1 \\ 0 & 2 & 1 & 5 \end{array} \right]$$

Step 2: Eliminate below the pivot in column 2 Use:

$$R_3 \leftarrow R_3 - 4R_2$$

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ 0 & 0.5 & 0.5 & 1 \\ 0 & 0 & -1 & 1 \end{array} \right]$$

Result: The matrix is now in upper triangular form.

6.3.2. Back-Substitution

From the upper triangular matrix we solve step by step:

$$\begin{aligned} -1 \cdot x_3 &= 1 &\Rightarrow & x_3 = -1 \\ 0.5x_2 + 0.5x_3 &= 1 &\Rightarrow & x_2 = 3 \\ 2x_1 + x_2 - x_3 &= 8 &\Rightarrow & x_1 = 2 \end{aligned}$$

Final solution:

$$x_1 = 2, \quad x_2 = 3, \quad x_3 = -1$$

6.3.3. Failures - No Unique Solution

The purpose of this illustration is to show what can happen if Gauss-Jordan fails. The computations will be done simultaneously on two linear systems:

$$\begin{aligned} x_1 + x_2 + x_3 &= 4, & x_1 + x_2 + x_3 &= 4, \\ 2x_1 + 2x_2 + x_3 &= 6, & \text{and} & \quad 2x_1 + 2x_2 + x_3 &= 4, \\ x_1 + x_2 + 2x_3 &= 6, & x_1 + x_2 + 2x_3 &= 6. \end{aligned}$$

These systems produce the augmented matrices

$$\tilde{A} = \left[\begin{array}{ccc|c} 1 & 1 & 1 & 4 \\ 2 & 2 & 1 & 6 \\ 1 & 1 & 2 & 6 \end{array} \right] \quad \text{and} \quad \tilde{A} = \left[\begin{array}{ccc|c} 1 & 1 & 1 & 4 \\ 2 & 2 & 1 & 4 \\ 1 & 1 & 2 & 6 \end{array} \right].$$

6. Linear Algebra

Since $a_{11} = 1$, we perform the row operations

$$R_2 \leftarrow R_2 - 2R_1, \quad R_3 \leftarrow R_3 - R_1$$

to obtain

$$\tilde{A} = \left[\begin{array}{ccc|c} 1 & 1 & 1 & 4 \\ 0 & 0 & -1 & -2 \\ 0 & 0 & 1 & 2 \end{array} \right] \quad \text{and} \quad \tilde{A} = \left[\begin{array}{ccc|c} 1 & 1 & 1 & 4 \\ 0 & 0 & -1 & -4 \\ 0 & 0 & 1 & 2 \end{array} \right].$$

At this point, $a_{22} = a_{32} = 0$. The algorithm requires that the procedure be halted, and no solution to either system is obtained. Writing the equations for each system gives:

$$\begin{array}{rcl} x_1 + x_2 + x_3 = 4, & & x_1 + x_2 + x_3 = 4, \\ -x_3 = -2, & \text{and} & -x_3 = -4, \\ x_3 = 2, & & x_3 = 2. \end{array}$$

The first linear system has an infinite number of solutions, which can be described by

$$x_3 = 2, \quad x_2 = 2 - x_1, \quad x_1 \text{ arbitrary.}$$

The second system leads to the contradiction $x_3 = 2$ and $x_3 = 4$, so no solution exists.

Conclusion: In each case, there is no *unique* solution.

6.3.4. Pivoting - Row swap

Sometimes, the system has a unique solution, but the order of rows simply does not allow for a direct application of the Gauss-Jordan elimination without reordering. Consider the example

$$A\vec{x} = \vec{b}, \quad A = \begin{bmatrix} 0 & 2 & 3 \\ 1 & -1 & 1 \\ 4 & 1 & -1 \end{bmatrix}, \quad \vec{b} = \begin{bmatrix} 5 \\ 2 \\ 1 \end{bmatrix}, \quad \vec{x} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

If you start elimination using row 1 as the pivot, you immediately face a problem because the first pivot element a_{11} is zero, so Gaussian elimination breaks down unless you swap rows.

Step 0 – pivoting (row swap)

Because $a_{11} = 0$, swap rows R_1 and R_3 :

$$\left[\begin{array}{ccc|c} 0 & 2 & 3 & 5 \\ 1 & -1 & 1 & 2 \\ 4 & 1 & -1 & 1 \end{array} \right] \xrightarrow{R_1 \leftrightarrow R_3} \left[\begin{array}{ccc|c} 4 & 1 & -1 & 1 \\ 1 & -1 & 1 & 2 \\ 0 & 2 & 3 & 5 \end{array} \right]$$

Step 1 – eliminate below the new pivot $a_{11} = 4$

$$m_{21} = \frac{1}{4}, \quad R_2 \leftarrow R_2 - m_{21} R_1$$

$$\left[\begin{array}{ccc|c} 4 & 1 & -1 & 1 \\ 0 & -\frac{5}{4} & \frac{5}{4} & \frac{7}{4} \\ 0 & 2 & 3 & 5 \end{array} \right]$$

Step 2 – eliminate below the pivot $a_{22} = -\frac{5}{4}$

$$m_{32} = \frac{2}{-\frac{5}{4}} = -\frac{8}{5} = -1.6, \quad R_3 \leftarrow R_3 - m_{32} R_2$$

$$\left[\begin{array}{ccc|c} 4 & 1 & -1 & 1 \\ 0 & -\frac{5}{4} & \frac{5}{4} & \frac{7}{4} \\ 0 & 0 & 5 & 7.8 \end{array} \right]$$

Now the system is upper-triangular.

Step 3 – back-substitution

$$z = \frac{7.8}{5} = 1.56,$$

$$-\frac{5}{4}y + \frac{5}{4}z = \frac{7}{4} \implies y = \frac{\frac{5}{4}z - \frac{7}{4}}{\frac{5}{4}} = 0.16,$$

$$4x + 1y - 1z = 1 \implies x = \frac{1 - y + z}{4} = 0.60.$$

Unique solution

$$x = 0.60, \quad y = 0.16, \quad z = 1.56$$

Without the initial row swap, the first pivot would be zero and the Gaussian elimination would fail—demonstrating the necessity of (partial) pivoting.

6.3.5. Algorithm

Here is the algorithm to solve the $n \times n$ linear system:

$$R_1: \quad a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = a_{1,n+1}$$

$$R_2: \quad a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = a_{2,n+1}$$

$$\vdots$$

$$R_n: \quad a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = a_{n,n+1}$$

6. Linear Algebra

INPUT: number of unknowns and equations n ; augmented matrix $A = [a_{ij}]$, where $1 \leq i \leq n$ and $1 \leq j \leq n + 1$.

OUTPUT: solution $x_1, x_2, \dots, x_n$ or message that the linear system has no unique solution.

Step 1: For $i = 1, \dots, n - 1$ do Steps 2–4. (*Elimination process.*)

Step 2: # check rows including i and below i

Let p be the smallest integer with $i \leq p \leq n$ and $a_{pi} \neq 0$.

If no integer p can be found then **OUTPUT** “no unique solution”;
STOP.

Step 3: If $p \neq i$ then perform $(R_p) \leftrightarrow (R_i)$.

Step 4: For $j = i + 1, \dots, n$ do Steps 5 and 6.

Step 5: Set $m_{ji} = a_{ji}/a_{ii}$.

Step 6: Perform $(R_j - m_{ji}R_i) \rightarrow (R_j)$.

Step 7: If $a_{nn} = 0$ then **OUTPUT** “no unique solution exists”;
STOP.

Step 8: Set $x_n = a_{n,n+1}/a_{nn}$. (*Start backward substitution.*)

Step 9: For $i = n - 1, \dots, 1$ set

$$x_i = \left(a_{i,n+1} - \sum_{j=i+1}^n a_{ij}x_j \right) / a_{ii}.$$

Step 10: **OUTPUT** $(x_1, \dots, x_n)$. (*Procedure completed successfully.*)
STOP.

6.3.6. Operation Counts

The time it takes to compute the solution and how much round-off errors accumulate depend on how many floating-point operations the computer has to perform. Multiplications and divisions usually take about the same amount of time, and they are typically significantly slower than additions or subtractions.

To get a sense of how many steps a method takes, we will count the operations needed to solve a standard system of n equations in n unknowns using the above algorithm. Since multiplications/divisions and additions/subtractions take different amounts of time, we count them separately.

Operation Counts for Gaussian Elimination (Steps 5–9)

No arithmetic operations are performed until Steps 5 and 6 in the algorithm.

Step 5 requires $(n - i)$ divisions.

Step 6 replaces the equation R_j by $(R_j - m_{ji}R_i)$, which requires:

- Multiplying m_{ji} with each term in R_i : $(n - i)(n - i + 1)$ multiplications,
- Subtracting the result from R_j : $(n - i)(n - i + 1)$ subtractions.

So, for each $i = 1, 2, \dots, n - 1$, the operation counts are:

Multiplications/divisions:

$$(n-i) + (n-i)(n-i+1) = (n-i)(n-i+2)$$

Additions/subtractions:

$$(n-i)(n-i+1)$$

Total Operation Counts (Steps 5–6)

We sum over $i = 1$ to $n - 1$. Recall from calculus:

$$\sum_{j=1}^m 1 = m, \quad \sum_{j=1}^m j = \frac{m(m+1)}{2}, \quad \sum_{j=1}^m j^2 = \frac{m(m+1)(2m+1)}{6}$$

Total Multiplications/Divisions:

$$\sum_{i=1}^{n-1} (n-i)(n-i+2) = \sum_{i=1}^{n-1} (n^2 - 2ni + i^2 + 2n - 2i) \quad (6.12)$$

$$= \frac{2n^3 + 3n^2 - 5n}{6} \quad (6.13)$$

Total Additions/Subtractions:

$$\sum_{i=1}^{n-1} (n-i)(n-i+1) = \sum_{i=1}^{n-1} (n-i)^2 + \sum_{i=1}^{n-1} (n-i) \quad (6.14)$$

$$= \frac{n^3 - n}{3} \quad (6.15)$$

Steps 8 and 9 (Backward Substitution)

Step 8 requires 1 division.

Step 9: For each $i = n - 1, \dots, 1$, we compute

$$x_i = \frac{a_{i,n+1} - \sum_{j=i+1}^n a_{ij}x_j}{a_{ii}}$$

This requires:

- $(n-i)$ multiplications,
- $(n-i-1)$ additions,
- 1 subtraction and 1 division.

The total cost of Steps 8 and 9 can be added similarly to the counts above.

6. Linear Algebra

Additional Multiplications/Divisions:

$$1 + \sum_{i=1}^{n-1} ((n-i) + 1) = 1 + \left(\sum_{i=1}^{n-1} (n-i) \right) + n - 1 \quad (6.16)$$

$$= \frac{n^2 + n}{2} \quad (6.17)$$

Additional Additions/Subtractions:

$$\sum_{i=1}^{n-1} ((n-i-1) + 1) = \sum_{i=1}^{n-1} (n-i) \quad (6.18)$$

$$= \frac{n^2 - n}{2} \quad (6.19)$$

Total Arithmetic Operations in Gauss-Jordan Algorithm:

Multiplications/Divisions:

$$\frac{2n^3 + 3n^2 - 5n}{6} + \frac{n^2 + n}{2} = \frac{n^3}{3} + n^2 - \frac{n}{3}$$

Additions/Subtractions:

$$\frac{n^3 - n}{3} + \frac{n^2 - n}{2} = \frac{n^3}{3} + \frac{n^2}{2} - \frac{5n}{6}$$

For large n , the total number of multiplications and divisions is approximately $\frac{n^3}{3}$, and similarly for additions and subtractions. This means the time and computational effort grow roughly with n^3 .

Table 6.1: Arithmetic Operation Counts for Selected Values of n

n	Multiplications/Divisions	Additions/Subtractions
3	17	11
10	430	375
50	44,150	42,875
100	343,300	338,250

6.4. Pivoting

6.4.1. General Idea

In deriving Gauss-Jordan, we found that a row interchange was needed when one of the pivot elements $a_{kk}^{(k)}$ is 0 in iteration (k) . This row interchange has the form $(R_k) \leftrightarrow (R_p)$,

where p is the smallest integer greater than k with $a_{pk}^{(k)} \neq 0$. To reduce round-off error, it is often necessary to perform row interchanges even when the pivot elements are not zero.

If $a_{kk}^{(k)}$ is small in magnitude compared to $a_{jk}^{(k)}$, then the magnitude of the multiplier

$$m_{jk} = \frac{a_{jk}^{(k)}}{a_{kk}^{(k)}}$$

will be much larger than 1. Round-off error introduced in the computation of one of the terms $a_{jk}^{(k)}$ is multiplied by m_{jk} when computing $a_{ji}^{(k+1)}$, which compounds the original error.

Also, when performing the backward substitution for

$$x_k = \frac{a_{k,n+1}^{(k)} - \sum_{j=k+1}^n a_{kj}^{(k)} x_j}{a_{kk}^{(k)}},$$

with a small value of $a_{kk}^{(k)}$, any error in the numerator can be dramatically increased because of the division by $a_{kk}^{(k)}$. In our next example, we will see that even for small systems, round-off error can dominate the calculations.

6.4.2. Example

Original System (no pivoting)

$$A = \begin{bmatrix} 10^{-10} & 1 \\ 1 & 1 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad A \begin{bmatrix} x \\ y \end{bmatrix} = b$$

Step 1: eliminate the lower entry in column 1

$$m = \frac{1}{10^{-10}} = 10^{10}, \quad R_2 \leftarrow R_2 - m \cdot R_1$$

$$\begin{bmatrix} 10^{-10} & 1 \\ 1 & 1 \end{bmatrix} \longrightarrow \begin{bmatrix} 10^{-10} & 1 \\ 0 & -10^{10} + 1 \approx -10^{10} \end{bmatrix}, \quad \begin{bmatrix} 1 \\ 2 \end{bmatrix} \longrightarrow \begin{bmatrix} 1 \\ -10^{10} + 2 \approx -10^{10} \end{bmatrix}$$

Step 2: back-substitution

$$-10^{10}y \approx -10^{10} \Rightarrow y \approx 1$$

$$10^{-10}x + y = 1 \Rightarrow x = \frac{1-y}{10^{-10}}$$

Here, the results of the computation can be twofold. If y is so close to unity that the code stores unity in the variable, then the resulting solution is $x = 0$. If y is not exactly

6. Linear Algebra

unity, but differs with a small round-off error ($y = 1 + \epsilon$), this error is random to some degree (depending on previous steps in the computation). The resulting value for x is then $x = 10^{10}\epsilon$, which is likely to have random character and the wrong result ($x \neq 1$). This solution tends to be numerically unstable.

With Pivoting (row swap first)

Swap the two rows:

$$A = \begin{bmatrix} 1 & 1 \\ 10^{-10} & 1 \end{bmatrix}, \quad b = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

Step 1: eliminate the lower entry in column 1

$$m = \frac{10^{-10}}{1} = 10^{-10}, \quad R_2 \leftarrow R_2 - m \cdot R_1$$

$$\begin{bmatrix} 1 & 1 \\ 10^{-10} & 1 \end{bmatrix} \longrightarrow \begin{bmatrix} 1 & 1 \\ 0 & 1 - 10^{-10} \end{bmatrix} \approx \begin{bmatrix} 1 & 1 \\ 0 & 0.9999999999 \end{bmatrix},$$

$$\begin{bmatrix} 2 \\ 1 \end{bmatrix} \longrightarrow \begin{bmatrix} 2 \\ 1 - 2 \cdot 10^{-10} = 0.9999999998 \end{bmatrix}$$

Step 2: back-substitution

$$0.9999999999 y = 0.9999999998 \Rightarrow y \approx 1 - 10^{-10}$$

$$x + y = 2 \Rightarrow x = 2 - y \approx 1 + 10^{-10}$$

$$x \approx 1 + 10^{-10}, \quad y \approx 1 - 10^{-10} \quad (\text{accurate and stable})$$

This example clearly shows how pivoting prevents catastrophic round-off errors and ensures meaningful solutions, especially for matrices with tiny or ill-scaled pivot entries.

Condition number for A

We note that the condition number of the matrix A_ϵ

$$A_\epsilon = \begin{bmatrix} 1 & 1 \\ \epsilon & 1 \end{bmatrix},$$

is not the problem of the unstable behaviour. We find for $\kappa_2(A_\epsilon)$ very similar values for different ϵ :

eps	kappa_2(A)
1.0e-02	2.653550e+00
1.0e-03	2.621550e+00
1.0e-04	2.618385e+00
1.0e-05	2.618069e+00
1.0e-06	2.618038e+00
1.0e-07	2.618034e+00
1.0e-08	2.618034e+00
1.0e-09	2.618034e+00
1.0e-10	2.618034e+00
1.0e-11	2.618034e+00
1.0e-12	2.618034e+00
1.0e-13	2.618034e+00
1.0e-14	2.618034e+00
1.0e-15	2.618034e+00
1.0e-16	2.618034e+00

6.5. Ill-conditioned matrices

In the above example the matrix A had strongly varying entries in absolute size, which made the order of the computation a crucial component. We already mentioned in the this chapter that the condition number of a matrix is also a measure for how strongly perturbations can grow. Since numerical computations and the resulting round-off errors occur everywhere, a large condition number can trigger an unstable behaviour even for matrices with coefficients of similar size.

6.5.1. Hilbert matrix

The Hilbert matrix of size $n \times n$ has elements

$$H_{ij} = \frac{1}{i+j-1}$$

It is symmetric, positive-definite, but very ill-conditioned as n increases. Consider the example for $n = 5$:

$$H_5 = \begin{pmatrix} 1 & \frac{1}{2} & \frac{1}{3} & \frac{1}{4} & \frac{1}{5} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} & \frac{1}{5} & \frac{1}{6} \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{5} & \frac{1}{6} & \frac{1}{7} \\ \frac{1}{4} & \frac{1}{5} & \frac{1}{6} & \frac{1}{7} & \frac{1}{8} \\ \frac{1}{5} & \frac{1}{6} & \frac{1}{7} & \frac{1}{8} & \frac{1}{9} \end{pmatrix} \quad (6.20)$$

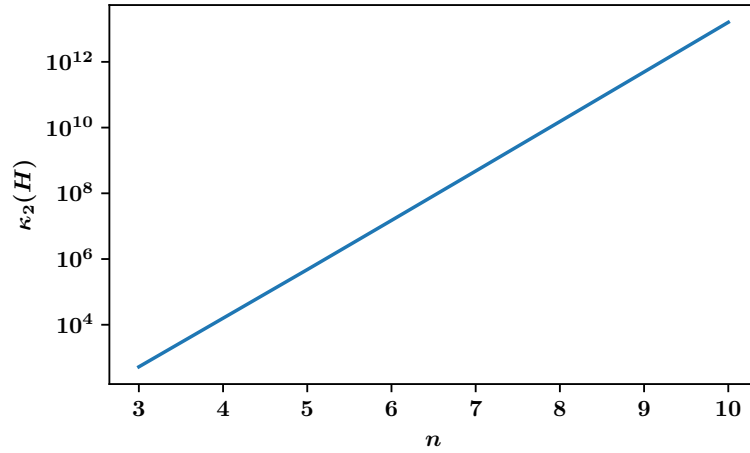


Figure 6.2.: Condition number for the Hilbert matrix. The numbers increase exponentially, so that for sizes larger than $n \approx 10$ the matrix becomes almost impossible to solve.

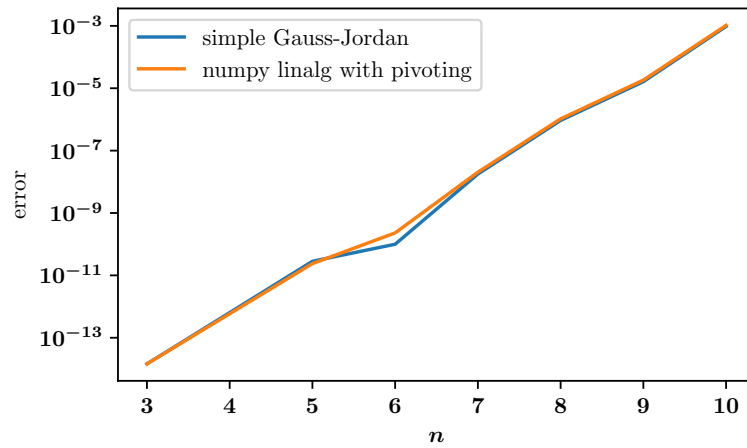


Figure 6.3.: Error for the solution of the Hilbert matrix with a known solution of $\vec{1}$. The error increases exponentially for both a simple Gauss-Jordan method as well as the powerful `numpy.linalg` methods including pivoting. A solution for $n > 10$ is basically not possible.

We note that all the values are of the same range. The condition number, however, increases exponentially with larger n , which is shown in Fig. 6.2. As a result, the error quickly increases, independent of whether pivoting is included or not. We address this by setting the *true* result to the unity vector $\vec{x} = \vec{1}$, then compute the right-hand side $\vec{b}$ via $H\vec{x} = \vec{b}$ and then apply the Gauss-Jordan method to H and $\vec{b}$. The error as a function of n is shown in Fig. 6.3 and indicates exponential growths similar to the condition number.

We note, that also the similar matrix given by

$$H'_{ij} = \frac{1}{i+j}$$

has the same problems. The condition number and the errors for solving the system increase exponentially with n . We highlight this example, because it appears in similar numbers in a variety of physics applications.

6.5.2. Examples of Matrices in Physics with Ill-Conditioned Structure

The following examples illustrate how matrices with challenging numerical properties, such as the Hilbert matrix, arise in real physics applications.

1. Radiative Transfer and Tomographic Inversion

In astrophysics and medical imaging, observed intensity is often modeled as a line-of-sight or projection integral over an emissivity function. This leads to Fredholm integral equations of the first kind, where discretization yields matrices similar to the Hilbert matrix:

$$I(y) = \int_0^1 \frac{f(x)}{x+y} dx \quad \Rightarrow \quad A_{ij} = \frac{1}{x_i + y_j}$$

These matrices are highly ill-conditioned, particularly for fine spatial grids.

2. Coupled Oscillators with Nearly Identical Parameters

In mechanical systems or lattice models, small differences in coupling constants or masses yield nearly singular stiffness matrices:

$$K = \begin{bmatrix} 2k & -k + \epsilon & 0 & \cdots \\ -k + \epsilon & 2k & -k + \epsilon & \cdots \\ \vdots & \ddots & \ddots & \ddots \end{bmatrix}$$

When $\epsilon \ll k$, the eigenvalues cluster tightly, causing numerical instabilities.

3. Capacitance Matrix in Thin Conducting Plates

In electrostatics, the mutual capacitance between thin, closely spaced conductors leads to matrices where:

$$C_{ij} \propto \frac{1}{|x_i - x_j|}$$

As the conductors become closely packed, the matrix becomes increasingly ill-conditioned, posing challenges for numerical inversion.

4. Degenerate Perturbation Theory in Quantum Mechanics

When treating nearly degenerate energy levels, the perturbed Hamiltonian matrix takes the form:

$$H = \begin{bmatrix} E & \epsilon & \epsilon \\ \epsilon & E + \delta & \epsilon \\ \epsilon & \epsilon & E - \delta \end{bmatrix}$$

with $\epsilon, \delta \ll E$. Small perturbations strongly affect eigenvalues and eigenvectors, highlighting sensitivity to numerical errors.

5. Gravitational Lensing Mass Inversion

In astrophysical gravitational lensing, the observed image positions are linked to the mass distribution via an integral kernel. Discretizing this system produces a matrix equation:

$$\vec{\alpha}_{\text{obs}} = A \cdot \vec{\kappa}$$

where $\vec{\alpha}_{\text{obs}}$ is the deflection field and $\vec{\kappa}$ is the mass distribution. The matrix A is often nearly singular due to overlapping deflection effects and limited resolution.

6.5.3. Gravitational Potential in 2D and Simplified Kernels

Gravitational Potential in Two Dimensions

We consider a planar distribution of mass with surface density $\rho(x, y)$, and compute the Newtonian gravitational potential Φ at a field point $(x', y') \in \mathbb{R}^2$. In two dimensions, the gravitational potential is given by:

$$\Phi(x', y') = -G \int_{\mathbb{R}^2} \frac{\rho(x, y)}{\sqrt{(x - x')^2 + (y - y')^2}} dx dy$$

Here, the kernel

$$K(x, y; x', y') = \frac{1}{\sqrt{(x - x')^2 + (y - y')^2}}$$

is the Green's function for the Laplace equation in 2D (up to a logarithmic factor that is absorbed in constants for planar gravity or electrostatics).

Simplification via Geometric Assumptions

To reduce this to a one-dimensional integral (for analytical or numerical study), we assume that the mass distribution is translationally symmetric in one direction — say, along the y -axis — so that:

$$\rho(x, y) = \rho(x)$$

Further, we evaluate the potential at a point along the y -axis, at height $y' > 0$, and set $x' = 0$ without loss of generality (by translational symmetry). The potential becomes:

$$\Phi(y') = -G \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \frac{\rho(x)}{\sqrt{x^2 + (y - y')^2}} dx dy \quad (6.21)$$

$$= -G \int_{-\infty}^{\infty} \rho(x) \left(\int_{-\infty}^{\infty} \frac{1}{\sqrt{x^2 + (y - y')^2}} dy \right) dx \quad (6.22)$$

This integral diverges in 2D due to the logarithmic behavior of the potential at infinity. In practice, we restrict the domain to a compact region. Alternatively, we compute the potential along a vertical line above a localized distribution on $x > 0$, so that:

$$\Phi(y') = -G \int_0^1 \frac{\rho(x)}{\sqrt{x^2 + y'^2}} dx$$

Here, y' plays the role of the vertical distance from the source layer, and the kernel becomes:

$$K(x, y') = \frac{1}{\sqrt{x^2 + y'^2}}.$$

This kernel has a different functional form as the discussed problematic one ($1/(x + y)$), but has similar scalings

- decays monotonically in both x and y ,
- shares the $\sim 1/r$ behavior with the true kernel,

and therefore has similar numerical values in the derived matrix. It is therefore likely that similarly high condition numbers apply.

6.5.4. Solution schemes for ill-conditioned matrices

Why Standard Solvers Fail

For ill-conditioned matrices:

- Round-off errors in finite-precision arithmetic are magnified.
- Pivoting (during LU decomposition, see below) is insufficient to stabilize the solution.
- Residuals may be small, but the solution vector $\vec{x}$ may still be highly inaccurate.

Robust Solution Techniques

To compute reliable solutions to $H\vec{x} = \vec{b}$, the following alternatives can be used:

6. Linear Algebra

(a) Higher Precision Arithmetic Use arbitrary-precision libraries (e.g., `mpmath` in Python) to avoid round-off errors:

```
mpmath.lu_solve(H, b)
```

This is the most accurate approach for small matrix sizes.

(b) Moore–Penrose Pseudoinverse The **Moore–Penrose pseudoinverse** A^+ is a generalization of the inverse that always exists, even for non-square or singular matrices ($\det(A) = 0$). It provides the best approximation (in the least-squares sense) to solving

$$A\vec{x} = \vec{b}$$

when an exact solution does not exist or is numerically unstable. The solution

$$\vec{x} = A^+\vec{b}$$

minimizes the norm $\|A\vec{x} - \vec{b}\|_2$, i.e., it is the vector $\vec{x}$ that makes $A\vec{x}$ as close as possible to $\vec{b}$ in the Euclidean norm (see parallel concept for fitting).

In practice, the pseudoinverse is computed via the Singular Value Decomposition (SVD):

6.6. Singular value decomposition (SVD)

The singular value decomposition (SVD) is one of the most important matrix factorizations in numerical linear algebra. It exists for every matrix, including non-square and ill-conditioned matrices.

6.6.1. Definition

For a real matrix

$$A \in \mathbb{R}^{m \times n}, \tag{6.23}$$

the singular value decomposition is

$$A = U\Sigma V^T, \tag{6.24}$$

where

- $U \in \mathbb{R}^{m \times m}$ is an orthogonal matrix,
- $V \in \mathbb{R}^{n \times n}$ is an orthogonal matrix,
- Σ is a diagonal matrix containing the singular values.

The singular values satisfy

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq 0. \quad (6.25)$$

The columns of U and V are orthonormal vectors,

$$U = (u_1, \dots, u_m), \quad V = (v_1, \dots, v_n). \quad (6.26)$$

The vectors u_i are called left singular vectors and the vectors v_i are called right singular vectors.

6.6.2. Geometric interpretation

The SVD can be interpreted as three successive operations:

$$\vec{x} \xrightarrow{V^T} \text{rotation/reflection} \xrightarrow{\Sigma} \text{stretching/compression} \xrightarrow{U} \text{rotation/reflection}. \quad (6.27)$$

The orthogonal matrices U and V^T only rotate or reflect the coordinate system and therefore preserve lengths and numerical stability. All stretching and compression is contained in the diagonal matrix Σ .

6.6.3. Ill-conditioned matrices

The SVD is especially useful for ill-conditioned matrices because it separates stable orthogonal transformations from unstable scaling. The condition number of a matrix can be written as

$$\kappa_2(A) = \frac{\sigma_{\max}}{\sigma_{\min}}. \quad (6.28)$$

If the smallest singular value $\sigma_{\min}$ becomes very small, the matrix becomes ill-conditioned. The inverse of A is formally

$$A^{-1} = V\Sigma^{-1}U^T, \quad (6.29)$$

with

$$\Sigma^{-1} = \begin{pmatrix} 1/\sigma_1 & & \\ & 1/\sigma_2 & \\ & & \ddots \end{pmatrix}. \quad (6.30)$$

Small singular values therefore produce large amplification factors:

$$\frac{1}{\sigma_i} \gg 1. \quad (6.31)$$

This explains why small perturbations in the data can lead to large changes in the solution of $A\vec{x} = \vec{b}$. In practice one sets

$$\Sigma_{ii}^{-1} = \begin{cases} 1/\sigma_i & \text{if } \sigma_i > \varepsilon \\ 0 & \text{otherwise} \end{cases} \quad (6.32)$$

This filtering of small σ_i acts as a form of regularization: it suppresses components of the solution corresponding to directions where A is nearly singular (i.e., where small errors in $\vec{b}$ could lead to large errors in $\vec{x}$).

6.6.4. Numerical stability

A major advantage of the SVD is that the orthogonal matrices U and V are numerically stable because they preserve norms:

$$\|U\vec{x}\|_2 = \|\vec{x}\|_2, \quad \|V\vec{x}\|_2 = \|\vec{x}\|_2. \quad (6.33)$$

As a result, numerical instability originates almost entirely from the small singular values in Σ rather than from the orthogonal transformations themselves. This makes the SVD one of the most robust methods for:

- solving ill-conditioned systems,
- computing pseudo-inverses,
- rank determination,
- noise filtering,
- least-squares fitting,
- dimensionality reduction.

6.7. Solution with LU Decomposition

6.7.1. General concept

For each $n \times n$ matrix A there is a decomposition in two triangular matrices of the form

$$L \cdot U = P \cdot A, \quad (6.34)$$

where L is a lower triangular matrix and U is an upper triangular matrix. The matrix P is the product of a sequence of permutation matrices, $P = P_1 \cdot P_2 \cdot P_3 \cdots$, which each exchange two rows.

If A is non-singular and Gaussian elimination works without row exchange, then $P = I$. In the following, we assume $P = I$ for simplicity, and so $L \cdot U = A$. For 4×4 matrices, this reads

$$\begin{pmatrix} \alpha_{11} & 0 & 0 & 0 \\ \alpha_{21} & \alpha_{22} & 0 & 0 \\ \alpha_{31} & \alpha_{32} & \alpha_{33} & 0 \\ \alpha_{41} & \alpha_{42} & \alpha_{43} & \alpha_{44} \end{pmatrix} \cdot \begin{pmatrix} \beta_{11} & \beta_{12} & \beta_{13} & \beta_{14} \\ 0 & \beta_{22} & \beta_{23} & \beta_{24} \\ 0 & 0 & \beta_{33} & \beta_{34} \\ 0 & 0 & 0 & \beta_{44} \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{pmatrix}. \quad (2.20)$$

We can now express $A \cdot \vec{x} = \vec{b}$ as

$$A \cdot \vec{x} = (L \cdot U) \cdot \vec{x} = L \cdot \underbrace{(U \cdot \vec{x})}_{\vec{y}} = \vec{b}, \quad (2.21)$$

and so we get two systems of equations,

$$L \cdot \vec{y} = \vec{b} \quad \text{and} \quad U \cdot \vec{x} = \vec{y}, \quad (2.22)$$

which we immediately can solve by substitution:

Forward substitutionfor $i = 1$ up to n :

$$\begin{aligned}
y_1 &= \frac{b_1}{\alpha_{11}} \\
&\vdots \\
y_i &= \frac{1}{\alpha_{ii}} \left(b_i - \sum_{j=1}^{i-1} \alpha_{ij} y_j \right).
\end{aligned} \tag{2.23}$$

Backward substitutionfor $i = n$ down to 1:

$$\begin{aligned}
x_n &= \frac{y_n}{\beta_{nn}} \\
&\vdots \\
x_i &= \frac{1}{\beta_{ii}} \left(y_i - \sum_{j=i+1}^n \beta_{ij} x_j \right)
\end{aligned} \tag{2.24}$$

6.7.2. Construction of LU

Gaussian elimination transforms A into an upper triangular matrix U by subtracting multiples of rows. Each elimination step corresponds to multiplication from the left by a lower triangular matrix. To eliminate entries below the pivot in column k , we use a matrix M_k of the form:

$$M_k = I - \vec{m}_k \vec{e}_k^T,$$

where:

- $\vec{e}_k$ is the k -th standard basis vector,
- $\vec{m}_k \in \mathbb{R}^n$ has zeros in the first k entries, and for $i > k$, $m_k^{(i)} = \frac{a_{ik}}{a_{kk}}$.

Then:

$$M_k A^{(k)} = A^{(k+1)},$$

and finally:

$$U = M_{n-1} \dots M_2 M_1 A.$$

We obtain:

$$A = M_1^{-1} M_2^{-1} \dots M_{n-1}^{-1} \cdot U = L \cdot U,$$

6. Linear Algebra

so

$$L = M_1^{-1} M_2^{-1} \dots M_{n-1}^{-1}. \quad (6.35)$$

Each inverse M_k^{-1} is also lower triangular and has the multipliers m_{ik} stored at position (i, k) .

6.7.3. LU Decomposition vs. Gauss–Jordan Elimination

Both LU decomposition and Gauss–Jordan elimination solve linear systems

$$A\vec{x} = \vec{b}, \quad (6.36)$$

but they differ significantly in computational cost, numerical stability, and practical applicability.

Basic idea

In LU decomposition, the matrix is factorized into $A = LU$. The system is then solved in two simple steps: $L\vec{y} = \vec{b}$ and $U\vec{x} = \vec{y}$. Gauss–Jordan elimination instead continues the elimination process until the matrix is transformed completely into the identity matrix:

$$[A|\vec{b}] \longrightarrow [I|\vec{x}]. \quad (6.37)$$

Thus

- LU decomposition stops after producing an upper triangular matrix,
- Gauss–Jordan additionally eliminates all entries above the pivots.

Computational cost

The LU factorization requires approximately $\frac{2}{3}N^3$ floating-point operations. Once the decomposition is known, solving the system only requires forward and backward substitution: $\mathcal{O}(N^2)$. Gauss–Jordan elimination requires approximately $\frac{4}{3}N^3$ operations because it eliminates matrix entries both below and above the pivots. Thus, LU decomposition is roughly twice as fast for large systems.

Multiple right-hand sides

A major advantage of LU decomposition is that the factorization can be reused. For systems

$$A\vec{x} = \vec{b}_1, \quad A\vec{x} = \vec{b}_2, \quad \dots \quad (6.38)$$

the expensive decomposition is computed only once. Each additional solution requires only triangular substitutions: $\mathcal{O}(N^2)$. Gauss–Jordan elimination, however, essentially repeats the elimination process for every new right-hand side.

Numerical stability

In practice, LU decomposition is usually combined with pivoting: $PA = LU$, where P is a permutation matrix. Pivoting avoids division by very small numbers, reduces roundoff amplification, and improves numerical stability. Gauss–Jordan elimination can also use pivoting, but since it performs more operations, it generally accumulates larger numerical errors.

Determinants and matrix inverses

LU decomposition also simplifies other matrix operations. For determinants:

$$\det(A) = \det(L) \det(U). \quad (6.39)$$

Since triangular determinants are simply products of diagonal entries,

$$\det(A) \approx \prod_i U_{ii}, \quad (6.40)$$

up to the sign introduced by pivoting. The inverse matrix can also be computed efficiently by solving

$$Ax_i = e_i \quad (6.41)$$

for all unit vectors e_i , while reusing the same LU factorization. Gauss–Jordan elimination can compute the inverse directly via

$$[A|I] \longrightarrow [I|A^{-1}], \quad (6.42)$$

but this is computationally more expensive.

Comparison

Property	LU decomposition	Gauss–Jordan
Operation count	$\frac{2}{3}N^3$	$\frac{4}{3}N^3$
Reuse for new b	excellent	poor
Triangular solves	yes	no
Determinant computation	simple	indirect
Matrix inverse	efficient	direct but expensive
Numerical stability	very good	worse
Practical use	standard method	mostly pedagogical

In modern scientific computing, LU decomposition is therefore preferred over Gauss–Jordan elimination for almost all large-scale applications.

6.8. Cholesky Decomposition

Definition

Let $A \in \mathbb{R}^{n \times n}$ be a symmetric, positive-definite matrix. The *Cholesky decomposition* is a factorization of the form

$$A = LL^T,$$

where $L \in \mathbb{R}^{n \times n}$ is a lower triangular matrix with positive diagonal entries.

Alternatively, the upper-triangular form $A = U^T U$ is sometimes used.

Conditions for Existence

The decomposition exists and is unique if and only if A is symmetric and positive-definite:

$$A = A^T, \quad \vec{x}^T A \vec{x} > 0 \quad \forall \vec{x} \neq \vec{0}.$$

Algorithm (Row-wise)

For $i = 1, \dots, n$, compute:

$$L_{ii} = \sqrt{A_{ii} - \sum_{k=1}^{i-1} L_{ik}^2},$$

$$L_{ij} = \frac{1}{L_{jj}} \left(A_{ij} - \sum_{k=1}^{j-1} L_{ik} L_{jk} \right), \quad \text{for } i > j.$$

Advantages

- Requires half the storage and computation compared to LU decomposition.
- Numerically more stable for symmetric positive-definite matrices.
- Faster than general LU: $\frac{1}{3}n^3$ vs. $\frac{2}{3}n^3$ flops.

Applications

- Solving $A\vec{x} = \vec{b}$ where A is symmetric positive-definite.
- Numerical optimization (e.g., Newton methods).
- Covariance matrix factorization in statistics and machine learning.

6.9. Householder Transformation

Definition

A *Householder transformation* is an orthogonal matrix $H \in \mathbb{R}^{n \times n}$ of the form

$$H = I - 2 \frac{\vec{v}\vec{v}^T}{\vec{v}^T\vec{v}},$$

where $\vec{v} \in \mathbb{R}^n \setminus \{0\}$ is a vector that defines the reflection hyperplane.

Geometric Meaning

H represents a reflection through a hyperplane perpendicular to $\vec{v}$. It can be used to zero out components of a vector below a certain position.

QR Decomposition via Householder

Given $A \in \mathbb{R}^{m \times n}$, the QR decomposition can be computed via Householder reflections:

$$A = QR,$$

where:

- $Q \in \mathbb{R}^{m \times m}$ is orthogonal ($Q^T Q = I$),
- $R \in \mathbb{R}^{m \times n}$ is upper triangular.

We construct a sequence of Householder matrices $H_1, H_2, \dots, H_k$ such that:

$$H_k \dots H_2 H_1 A = R, \quad \text{and} \quad Q = H_1 H_2 \dots H_k.$$

Constructing the Vector $\vec{v}$

To zero out entries below position i in column i , define:

$$\vec{v} = \vec{x} + \text{sign}(x_1) \|\vec{x}\|_2 \vec{e}_1,$$

where $\vec{x}$ is the target column segment and $\vec{e}_1$ is the first unit vector. This choice ensures numerical stability.

Advantages

- High numerical stability due to use of orthogonal transformations.
- Basis for many least-squares solvers.

Applications

- Least-squares problems (especially when A is tall).
- Eigenvalue algorithms
- QR factorization in numerical linear algebra libraries.

6.10. Iterative Method

When solving a system of linear equations on a computer, we frequently have the problem that we accumulate truncation errors which can lead to undesired uncertainties and errors in the solution vector. In order to reach machine precision, iteration is required.

Consider once again the system

$$A\vec{x} = \vec{b}$$

and assume that your solver so far has obtained an estimate of the solution as $(\vec{x} + \delta\vec{x})$, where the term $\delta\vec{x}$ denotes the unknown error. Let us multiply with A from the left to get

$$A(\vec{x} + \delta\vec{x}) = A\vec{x} + A\delta\vec{x} = \vec{b} + \delta\vec{b}, \quad (6.43)$$

where we identify $A\delta\vec{x} = \delta\vec{b}$. With that, we can rearrange this to get

$$A\delta\vec{x} = A(\vec{x} + \delta\vec{x}) - \vec{b}, \quad (6.44)$$

where all terms on the right-hand side are known or can be calculated. We can solve for $\delta\vec{x}$ by multiplying with the inverse A^{-1} from the left,

$$\delta\vec{x} = (\vec{x} + \delta\vec{x}) - A^{-1}\vec{b}. \quad (6.45)$$

We can now subtract $\delta\vec{x}$ from our original solution $(\vec{x} + \delta\vec{x})$ to obtain a better estimate of the true solution $\vec{x}$.